UD3 - Consultas SQL (III)

Nesta parte da unidade didáctica estudaremos como combinar información procedente de varias táboas mediante composicións internas e externas. Aprenderemos a relacionar tres ou máis táboas, aplicar condicións de filtrado e identificar rexistros que non teñen correspondencia noutras táboas.

13. Composicións internas con varias táboas

13.1. Recuperar información de varias táboas

Nunha base de datos relacional, a información adoita estar distribuída en diferentes táboas. Para obter información completa é necesario combinar os rexistros relacionados mediante as súas claves.

A instrución `INNER JOIN` permite combinar dúas táboas utilizando unha condición de unión. Cando necesitamos información de máis de dúas táboas, podemos encadear varias composicións `INNER JOIN`.

> **Idea clave:** cada `JOIN` incorpora unha nova táboa ao resultado, utilizando unha condición que relaciona os seus rexistros cos das táboas xa incluídas na consulta.

13.2. Sintaxe con varias táboas

```
SELECT columnas
FROM taboa1
INNER JOIN taboa2
    ON taboa1.clave = taboa2.clave
INNER JOIN taboa3
    ON taboa2.clave = taboa3.clave;
```

Cada composición debe incluír a súa propia condición `ON`, que indica como se relacionan as dúas partes da consulta.

13.3. Exemplo: alumnos e cursos

Supoñamos que temos dúas táboas relacionadas:

- `cursos`: almacena os cursos dispoñibles.
- `alumnos`: almacena os alumnos e o curso ao que pertencen.

A relación establécese mediante o campo `curso_id` da táboa `alumnos`, que referencia o campo `id` da táboa `cursos`.

```
SELECT alumnos.nombre,
       cursos.nombre AS curso
FROM alumnos
INNER JOIN cursos
    ON alumnos.curso_id = cursos.id;
```

O resultado mostra o nome de cada alumno xunto co nome do curso no que está matriculado. Só aparecen os alumnos que teñen un curso correspondente.

13.4. Uso de alias nas táboas

Cando unha consulta utiliza varias táboas, podemos asignarlles alias para simplificar a escritura e mellorar a súa lectura. O alias defínese inmediatamente despois do nome da táboa.

```
SELECT a.nombre,
       c.nombre AS curso
FROM alumnos AS a
INNER JOIN cursos AS c
    ON a.curso_id = c.id;
```

Neste exemplo:

- `a` é o alias da táboa `alumnos`.
- `c` é o alias da táboa `cursos`.
- `AS curso` é o alias da columna no resultado.

> **Importante:** cando unha columna existe en varias táboas, é recomendable indicar sempre a táboa ou o seu alias. Por exemplo, `a.id` e `c.id` permiten distinguir os dous campos.

13.5. Condicións de unión

A condición de unión establece que rexistros deben combinarse. Normalmente utiliza unha clave primaria e unha clave foránea.

```
SELECT a.nombre,
       c.nombre AS curso
FROM alumnos a
INNER JOIN cursos c
    ON a.curso_id = c.id;
```

A condición `ON a.curso_id = c.id` indica que cada alumno se combina co curso cuxo identificador coincide co valor almacenado na súa clave foránea.

13.6. Diferenza entre ON e WHERE

A cláusula `ON` establece a relación entre as táboas. A cláusula `WHERE` filtra os rexistros que forman parte do resultado.

```
SELECT a.nombre,
       c.nombre AS curso
FROM alumnos a
INNER JOIN cursos c
    ON a.curso_id = c.id
WHERE c.nombre = 'DAW1';
```

A consulta combina alumnos e cursos e, posteriormente, mostra unicamente os alumnos pertencentes ao curso DAW1.

13.7. Exercicios de consolidación

1. Obtén o nome de todos os alumnos xunto co nome do seu curso.
2. Mostra o identificador e o nome dos alumnos xunto co identificador e o nome do curso.
3. Utiliza alias para abreviar os nomes das táboas.
4. Obtén os alumnos que pertencen ao curso DAM1.
5. Mostra os alumnos ordenados polo nome do curso e polo nome do alumno.

14. Composicións internas con tres ou máis táboas

14.1. Relacionar varias táboas

Nas bases de datos reais é habitual que a información necesaria para unha consulta estea distribuída en tres ou máis táboas. Podemos combinar todas elas mediante varias cláusulas `INNER JOIN`.

Cada nova composición debe indicar a condición que permite relacionar a táboa incorporada coas táboas anteriores.

14.2. Exemplo con tres táboas

Imos utilizar un exemplo cunha base de datos que almacena información sobre alumnos, matrículas e módulos.

- `Alumno`: datos dos alumnos.
- `Matricula`: relación entre alumnos e módulos, incluíndo a cualificación.
- `Modulo`: información dos módulos formativos.

A relación entre as táboas é a seguinte:

- `Matricula.id_alumno` referencia `Alumno.id`.
- `Matricula.id_modulo` referencia `Modulo.id`.

```
SELECT a.nombre,
       m.nombre AS modulo,
       ma.nota
FROM Alumno a
INNER JOIN Matricula ma
    ON a.id = ma.id_alumno
INNER JOIN Modulo m
    ON ma.id_modulo = m.id;
```

A consulta combina as tres táboas para obter o nome do alumno, o módulo no que está matriculado e a nota obtida.

14.3. Engadir condicións de filtrado

Podemos combinar os `JOIN` con condicións `WHERE` para seleccionar unicamente os rexistros que cumpren determinados requisitos.

```
SELECT a.nombre,
       m.nombre AS modulo,
       ma.nota
FROM Alumno a
INNER JOIN Matricula ma
    ON a.id = ma.id_alumno
INNER JOIN Modulo m
    ON ma.id_modulo = m.id
WHERE ma.nota >= 5;
```

Neste caso, só se mostran as matrículas nas que o alumno obtivo unha nota igual ou superior a 5.

14.4. Combinar filtros

Podemos utilizar os operadores lóxicos para aplicar varias condicións á vez.

```
SELECT a.nombre,
       m.nombre AS modulo,
       ma.nota
FROM Alumno a
INNER JOIN Matricula ma
    ON a.id = ma.id_alumno
INNER JOIN Modulo m
    ON ma.id_modulo = m.id
WHERE ma.nota >= 5
  AND m.nombre = 'Bases de datos';
```

A consulta mostra os alumnos que aprobaron o módulo Bases de datos.

14.5. Ordenar os resultados

A cláusula `ORDER BY` permite ordenar os resultados dunha consulta que combina varias táboas. Podemos ordenar por columnas de calquera das táboas incluídas na composición.

```
SELECT a.nombre,
       m.nombre AS modulo,
       ma.nota
FROM Alumno a
INNER JOIN Matricula ma
    ON a.id = ma.id_alumno
INNER JOIN Modulo m
    ON ma.id_modulo = m.id
ORDER BY m.nombre ASC,
         ma.nota DESC;
```

Primeiro ordénanse os resultados polo nome do módulo e, dentro de cada módulo, pola nota de maior a menor.

14.6. Exemplo completo

Queremos obter os alumnos, os módulos nos que están matriculados e as súas notas. Só deben aparecer as notas superiores ou iguais a 5, ordenadas por módulo e alumno.

```
SELECT a.nombre AS alumno,
       m.nombre AS modulo,
       ma.nota
FROM Alumno a
INNER JOIN Matricula ma
    ON a.id = ma.id_alumno
INNER JOIN Modulo m
    ON ma.id_modulo = m.id
WHERE ma.nota >= 5
ORDER BY m.nombre ASC,
         a.nombre ASC;
```

14.7. Exercicios de consolidación

1. Mostra o nome dos alumnos, os módulos nos que están matriculados e as súas notas.
2. Obtén os alumnos que teñen unha nota superior ou igual a 7.
3. Mostra unicamente as matrículas dun módulo determinado.
4. Obtén os alumnos aprobados nun módulo concreto, ordenados pola nota de maior a menor.
5. Mostra todas as matrículas ordenadas primeiro polo nome do alumno e despois polo nome do módulo.

15. Composicións externas: LEFT JOIN

15.1. Limitacións de INNER JOIN

A composición interna `INNER JOIN` devolve unicamente os rexistros que teñen correspondencia nas dúas táboas. Isto significa que os rexistros sen relación quedan excluídos do resultado.

Por exemplo, se temos un curso que aínda non ten alumnos, unha consulta con `INNER JOIN` entre `cursos` e `alumnos` non mostrará ese curso.

15.2. Que é LEFT JOIN?

A composición externa esquerda, `LEFT JOIN`, devolve:

- Todos os rexistros da táboa situada á esquerda do JOIN.
- Os rexistros correspondentes da táboa situada á dereita.
- Valores `NULL` nas columnas da táboa dereita cando non existe correspondencia.

> **Idea clave:** `LEFT JOIN` conserva todos os rexistros da primeira táboa, aínda que non teñan unha relación coa segunda.

15.3. Sintaxe

```
SELECT columnas
FROM taboa_esquerda
LEFT JOIN taboa_dereita
    ON taboa_esquerda.clave = taboa_dereita.clave;
```

15.4. Exemplo: todos os cursos e os seus alumnos

Queremos mostrar todos os cursos, incluídos aqueles nos que aínda non hai alumnos matriculados.

```
SELECT c.nombre AS curso,
       a.nombre AS alumno
FROM cursos c
LEFT JOIN alumnos a
    ON c.id = a.curso_id;
```

O resultado incluirá todos os cursos. Se un curso non ten alumnos, a columna `alumno` aparecerá con valor `NULL`.

15.5. Interpretación dos resultados

Nun resultado de `LEFT JOIN` podemos atopar dous tipos de rexistros:

- Rexistros con correspondencia: aparecen os datos das dúas táboas.
- Rexistros sen correspondencia: aparecen os datos da táboa esquerda e valores `NULL` na táboa dereita.

Os valores `NULL` non indican necesariamente un erro. Significan que non existe un rexistro relacionado na táboa dereita.

15.6. Diferencia entre INNER JOIN e LEFT JOIN

```
-- Só cursos con alumnos
SELECT c.nombre, a.nombre
FROM cursos c
INNER JOIN alumnos a
    ON c.id = a.curso_id;

-- Todos os cursos, teñan ou non alumnos
SELECT c.nombre, a.nombre
FROM cursos c
LEFT JOIN alumnos a
    ON c.id = a.curso_id;
```

A diferenza está nos rexistros que se conservan: o `INNER JOIN` esixe correspondencia, mentres que o `LEFT JOIN` conserva todos os rexistros da táboa esquerda.

15.7. LEFT JOIN con varias táboas

Tamén podemos combinar varias táboas mediante composicións externas.

```
SELECT c.nombre AS curso,
       a.nombre AS alumno,
       ma.nota
FROM cursos c
LEFT JOIN alumnos a
    ON c.id = a.curso_id
LEFT JOIN Matricula ma
    ON a.id = ma.id_alumno;
```

A consulta conserva todos os cursos. Se un curso non ten alumnos, ou se un alumno non ten matrículas, as columnas correspondentes aparecerán con valores `NULL`.

15.8. Exercicios de consolidación

1. Mostra todos os cursos e os alumnos asociados a cada un.
2. Inclúe no resultado os cursos que non teñen alumnos.
3. Mostra todos os cursos e, cando existan, os alumnos e as súas matrículas.
4. Compara o resultado dunha consulta con `INNER JOIN` e outra con `LEFT JOIN`.

16. Composicións externas: RIGHT JOIN

16.1. Que é RIGHT JOIN?

A composición externa dereita, `RIGHT JOIN`, funciona de maneira similar a `LEFT JOIN`, pero conserva todos os rexistros da táboa situada á dereita da composición.

Devolve:

- Todos os rexistros da táboa dereita.
- Os rexistros correspondentes da táboa esquerda.
- Valores `NULL` nas columnas da táboa esquerda cando non existe correspondencia.

16.2. Sintaxe

```
SELECT columnas
FROM taboa_esquerda
RIGHT JOIN taboa_dereita
    ON taboa_esquerda.clave = taboa_dereita.clave;
```

16.3. Exemplo: todos os alumnos e os seus cursos

Imos utilizar unha composición dereita entre as táboas `cursos` e `alumnos`.

```
SELECT c.nombre AS curso,
       a.nombre AS alumno
FROM cursos c
RIGHT JOIN alumnos a
    ON c.id = a.curso_id;
```

Esta consulta conserva todos os alumnos. Se algún alumno non ten un curso correspondente, as columnas da táboa `cursos` aparecerán con valor `NULL`.

16.4. Comparación entre LEFT JOIN e RIGHT JOIN

A principal diferenza entre ambas as composicións é a táboa cuxos rexistros se conservan.

```
-- Conserva todos os cursos
SELECT c.nombre, a.nombre
FROM cursos c
LEFT JOIN alumnos a
    ON c.id = a.curso_id;

-- Conserva todos os alumnos
SELECT c.nombre, a.nombre
FROM cursos c
RIGHT JOIN alumnos a
    ON c.id = a.curso_id;
```

As dúas consultas parten da mesma relación, pero conservan táboas diferentes.

16.5. Detección de rexistros sen correspondencia

Un dos usos máis interesantes das composicións externas é localizar rexistros que non teñen relación con outros.

Para iso, realizamos un `LEFT JOIN` ou `RIGHT JOIN` e filtramos os resultados nos que a columna correspondente da outra táboa ten valor `NULL`.

16.6. Exemplo: cursos sen alumnos

Queremos localizar os cursos nos que non hai ningún alumno.

```
SELECT c.id,
       c.nombre
FROM cursos c
LEFT JOIN alumnos a
    ON c.id = a.curso_id
WHERE a.id IS NULL;
```

A consulta conserva inicialmente todos os cursos. Despois, a condición `WHERE a.id IS NULL` selecciona unicamente aqueles para os que non se atopou ningún alumno.

16.7. Exemplo: alumnos sen curso asociado

Tamén podemos detectar alumnos que non teñen un curso correspondente.

```
SELECT a.id,
       a.nombre
FROM alumnos a
LEFT JOIN cursos c
    ON a.curso_id = c.id
WHERE c.id IS NULL;
```

Esta consulta mostra os alumnos para os que non existe unha correspondencia na táboa `cursos`.

> **Importante:** para comprobar se existe correspondencia, debemos utilizar unha columna da táboa que podería non ter rexistro asociado e que normalmente non admite valores `NULL`, como a súa clave primaria.

16.8. LEFT JOIN ou RIGHT JOIN?

Calquera consulta que utilice `RIGHT JOIN` pode normalmente escribirse utilizando `LEFT JOIN`, cambiando a orde das táboas.

```
-- Usando RIGHT JOIN
SELECT c.nombre, a.nombre
FROM cursos c
RIGHT JOIN alumnos a
    ON c.id = a.curso_id;

-- Consulta equivalente usando LEFT JOIN
SELECT c.nombre, a.nombre
FROM alumnos a
LEFT JOIN cursos c
    ON a.curso_id = c.id;
```

Ambas as consultas conservan todos os alumnos e incorporan a información do curso cando existe correspondencia.

16.9. Exercicios de consolidación

1. Mostra todos os alumnos e o nome do seu curso mediante `RIGHT JOIN`.
2. Obtén os alumnos que non teñen un curso asociado.
3. Reescribe unha consulta con `RIGHT JOIN` utilizando `LEFT JOIN`.
4. Compara os resultados das dúas consultas.

17. Consultas con composicións externas e condicións de filtrado

17.1. Combinar JOIN e WHERE

As composicións externas poden combinarse con condicións de filtrado para seleccionar os rexistros que cumpren determinados requisitos.

Non obstante, é importante comprender como afectan as condicións `WHERE` ao resultado dunha composición externa, especialmente cando existen valores `NULL`.

17.2. Filtrar os resultados dun LEFT JOIN

Supoñamos que queremos mostrar todos os cursos e os alumnos que pertencen ao curso DAW1.

```
SELECT c.nombre AS curso,
       a.nombre AS alumno
FROM cursos c
LEFT JOIN alumnos a
    ON c.id = a.curso_id
WHERE c.nombre = 'DAW1';
```

A consulta conserva a lóxica do `LEFT JOIN` para o curso DAW1: se non ten alumnos, seguirá aparecendo cun valor `NULL` na columna correspondente.

17.3. Filtrar a táboa dereita utilizando ON

Existe unha diferenza importante entre colocar unha condición na cláusula `ON` ou na cláusula `WHERE`.

Queremos mostrar todos os cursos e só os alumnos maiores de idade asociados a cada curso.

```
SELECT c.nombre AS curso,
       a.nombre AS alumno,
       a.edad
FROM cursos c
LEFT JOIN alumnos a
    ON c.id = a.curso_id
   AND a.edad >= 18;
```

Neste caso, todos os cursos aparecen no resultado. Para cada curso só se incorporan os alumnos que teñen 18 anos ou máis. Os cursos sen alumnos que cumpran esa condición aparecen igualmente, coas columnas de alumno a `NULL`.

17.4. Diferenza entre ON e WHERE nunha composición externa

Compara as seguintes consultas:

```
-- Consulta A: condición na cláusula ON
SELECT c.nombre AS curso,
       a.nombre AS alumno
FROM cursos c
LEFT JOIN alumnos a
    ON c.id = a.curso_id
   AND a.edad >= 18;

-- Consulta B: condición na cláusula WHERE
SELECT c.nombre AS curso,
       a.nombre AS alumno
FROM cursos c
LEFT JOIN alumnos a
    ON c.id = a.curso_id
WHERE a.edad >= 18;
```

A primeira consulta conserva todos os cursos, aínda que non teñan alumnos maiores de idade.

A segunda consulta elimina os rexistros nos que `a.edad` é `NULL`, porque a condición `a.edad >= 18` non se cumpre para eses valores. Polo tanto, pode comportarse como unha composición interna respecto dese filtro.

> **Regra práctica:** utiliza `ON` para definir a relación e as condicións que deben cumprir os rexistros da táboa relacionada. Utiliza `WHERE` para filtrar o resultado final. Se queres conservar os rexistros sen correspondencia, comproba que as condicións de `WHERE` non os eliminen.

17.5. Detectar rexistros sen correspondencia con filtros

Podemos combinar unha composición externa con condicións para detectar rexistros que non teñen relación ou que non cumpren determinados requisitos.

Por exemplo, queremos localizar os cursos que non teñen alumnos maiores de idade:

```
SELECT c.id,
       c.nombre
FROM cursos c
LEFT JOIN alumnos a
    ON c.id = a.curso_id
   AND a.edad >= 18
WHERE a.id IS NULL;
```

A condición incluída en `ON` limita os alumnos considerados aos maiores de idade. A condición `WHERE a.id IS NULL` selecciona os cursos nos que non se atopou ningún alumno que cumpra ese requisito.

17.6. Combinar LEFT JOIN con GROUP BY

Tamén podemos utilizar composicións externas xunto con funcións de agregación para contar rexistros relacionados, incluíndo aqueles que non teñen correspondencia.

Queremos mostrar todos os cursos e o número de alumnos asociados a cada un.

```
SELECT c.id,
       c.nombre AS curso,
       COUNT(a.id) AS total_alumnos
FROM cursos c
LEFT JOIN alumnos a
    ON c.id = a.curso_id
GROUP BY c.id, c.nombre;
```

A consulta mostra todos os cursos. Os cursos sen alumnos aparecen cun total de cero.

> **Atención:** neste exemplo utilizamos `COUNT(a.id)` e non `COUNT(*)`. Nun `LEFT JOIN`, un curso sen alumnos produce igualmente unha fila no resultado intermedio, pero `a.id` terá valor `NULL`. `COUNT(a.id)` non conta ese valor, polo que devolve cero.

17.7. Exercicios prácticos

Resolve os seguintes exercicios utilizando as táboas `cursos` e `alumnos`:

1. Mostra todos os cursos xunto cos alumnos asociados, incluíndo os cursos sen alumnos.
2. Obtén os cursos que non teñen ningún alumno.
3. Mostra todos os cursos e os alumnos maiores de idade asociados a cada un, sen eliminar os cursos que non teñan alumnos que cumpran esa condición.

4. Obtén os cursos que non teñen ningún alumno maior de idade.
5. Mostra o número de alumnos de cada curso, incluíndo os cursos sen alumnos.
6. Obtén todos os cursos e o número de alumnos maiores de idade asociados a cada un.
7. Mostra os cursos que teñen máis de dous alumnos.
8. Obtén os cursos que non teñen alumnos e ordénaos polo nome.

17.8. Resumo das composicións externas

Tipo de JOIN	Rexistros que conserva	Uso habitual
`INNER JOIN`	Só os que teñen correspondencia nas dúas táboas.	Combinar información relacionada.
`LEFT JOIN`	Todos os rexistros da táboa esquerda.	Incluír rexistros sen correspondencia na táboa dereita.
`RIGHT JOIN`	Todos os rexistros da táboa dereita.	Incluír rexistros sen correspondencia na táboa esquerda.

As composicións externas permiten conservar información que desaparecería nunha composición interna. Son especialmente útiles para localizar rexistros sen relación, realizar informes completos e calcular totais que inclúan categorías sen rexistros asociados.